JavaScript Fundamentals

Beginner-friendly lecture notes • Based on the October 23 class • Examples, explanations, practice and revision

How to use these notes: Read the idea first, then type the code yourself in an HTML file and open it in Chrome. For console output, open DevTools with **Ctrl + Shift + J**.

1. What is JavaScript?

JavaScript (JS) adds behaviour and interactivity to a webpage. HTML provides structure, CSS controls appearance, and JavaScript can make the page respond to actions or change content.

The lecture describes JavaScript as interpreted by the browser's JavaScript engine and as not being a type-safe language. TypeScript is mentioned as an option that adds static type checking. Modern JavaScript engines may internally compile code too, so “interpreter only” is a beginner-level simplification.

Simple comparison: HTML = structure; CSS = design; JavaScript = behaviour.

2. Where do we write JavaScript?

Write JavaScript inside a **<script>** tag. The lecture recommends placing the script near the end of the **<body>**, so the page's HTML is already present when the script runs.

```
<html>
    <head>
        <title>JS Demo</title>
    </head>
    <body>
        <h1>JavaScript Demo</h1>

        <script>
            document.write("hello world");
            console.log("hello console");
            window.alert("hii user");
        </script>
    </body>
</html>
```

What each line does

document.write(...) writes content into the webpage while the page is being parsed.

console.log(...) prints a message in the browser's developer console. It is useful for checking values and debugging.

window.alert(...) opens a browser alert popup.

Try the three lines separately at first so you can see where each output appears.

3. Three useful predefined objects

Object	Represents / use	Example
document	The webpage document; used for webpage-related work	document.write("Hello");
console	The developer console; useful for output and debugging	console.log("Hello");
window	The browser window; provides browser-window features	window.alert("Hi!");

Important wording: In beginner lessons, you may hear “everything in JavaScript is an object.” Treat this as a reminder that JS can work with webpage elements and browser objects. Strictly speaking, JavaScript values also include primitive

types such as numbers, strings, booleans, undefined, null, bigint and symbols; not everything is an object.

4. Variables and dynamic data types

A variable stores a value. JavaScript variables do not require you to write a type such as int or String when declaring them. The type is determined from the value at runtime, and a variable can later be assigned a value of another type.

```
let a = 4;
console.log(typeof a);   // number
console.log(a);          // 4

a = "hello";
console.log(typeof a);   // string
console.log(a);          // hello

let b = true;
console.log(typeof b);   // boolean
console.log(b);          // true

let obj = null;
console.log(typeof obj); // object (historical JS behaviour)

let obj1;
console.log(typeof obj1); // undefined
```

Remember: JavaScript uses **number** for ordinary numeric values, including whole numbers and decimals. The result of **typeof null** is the historically surprising string **"object"**.

Common types from this lesson

Value	typeof result
4	"number"
"hello"	"string"
true	"boolean"
undefined	"undefined"
null	"object" (historical behaviour)

5. Operators

Arithmetic operators

Used for mathematical calculations.

```
console.log(7 + 3);   // 10
console.log(7 - 3);   // 4
console.log(7 * 3);   // 21
console.log(7 / 2);   // 3.5
console.log(7 % 2);   // 1 (remainder)
console.log(2 ** 3);  // 8 (power)
```

String concatenation and template literals

The + operator can join strings. Template literals use backticks and ${...} to insert values into a string.

```
let a = 4;
let b = 6;
let c = a + b;
console.log(`sum of ${a} and ${b} is ${c}`);
// sum of 4 and 6 is 10
```

If one side of + is a string, JavaScript commonly concatenates. Other arithmetic operators may convert numeric strings to numbers. Invalid numeric operations can produce **NaN**, meaning “Not a Number.”

```
console.log("5" + 2);    // "52"
```

```
console.log("5" - 2);    // 3
console.log("abc" - 2);  // NaN
```

Relational, equality and logical operators

Group	Operators	Purpose
Relational	<, >, <=, >=	Compare magnitude
Equality	==, !=, ===	Compare values; === also checks type
Logical	||, &&, !	OR, AND, NOT
Conditional	condition ? value1 : value2	Choose one of two values

```
let a = 70;
let b = 160;
let c = a > b ? a : b;
console.log(c); // 160
```

Read the conditional expression as: if a > b is true, choose a; otherwise choose b. Prefer === for equality checks when you want the value and type to match.

6. if, if-else, else-if ladder and switch

if-else

```
let age = 20;

if (age >= 18) {
    console.log("Eligible");
} else {
    console.log("Not eligible");
}
```

else-if ladder: multiple conditions

```
let id = 102;

if (id === 101) {
    console.log("Rohan");
} else if (id === 102) {
    console.log("Sohan");
} else if (id === 103) {
    console.log("Mohan");
} else {
    console.log("Not found");
}
```

The conditions are checked from top to bottom. When one condition is true, its block runs and the rest of this ladder is skipped.

switch

```
let id = 103;

switch (id) {
    case 101:
        console.log("Rohan");
        break;
    case 102:
        console.log("Sohan");
        break;
    case 103:
        console.log("Mohan");
        break;
    default:
        console.log("Not found");
}
```

break stops execution from falling through into the next case. **default** runs when no case matches.

Use if-else for conditions/ranges and switch when one expression is compared against several specific values.

7. Loops

while loop

```
let i = 1;

while (i <= 10) {
    console.log(i);
    i++;
}
```

The condition is checked before each iteration. The loop stops when the condition becomes false. If you forget to update the variable, the loop may never stop.

for loop

```
for (let i = 1; i <= 10; i++) {
    console.log(i);
}
```

The three parts are: initialization; condition; increment/decrement.

Nested loops: print 10 rows of 5 stars

```
for (let j = 1; j <= 10; j++) {
    for (let i = 1; i <= 5; i++) {
        document.write("*");
    }
    document.write("<br>");
}
```

The inner loop prints five stars on one row. The outer loop repeats that whole row ten times. **
** starts a new line in the webpage.

Tip: **console.log** prints in DevTools; **document.write** writes to the webpage. Use the one that matches the output requested.

8. Functions

A function is a group of statements that performs a task. Functions help with **reusability** (use the same code more than once) and **modularity** (split a large problem into smaller parts).

```
function add(n1, n2) {
    let n3 = n1 + n2;
    return n3;
}

let result = add(4, 8);
console.log(result); // 12
```

When add(4, 8) is called, 4 is passed into n1 and 8 into n2. The function adds them, returns 12, and the returned value is stored in result.

Single responsibility: a function should ideally do one clear job. For example, add should calculate and return a sum; the caller decides whether to print it, save it, or use it elsewhere.

JavaScript function syntax does not specify a return type or argument data types. A function may return a value with return, or omit return if no result needs to be sent back.

9. var, let and const

The lecture introduces three keywords for declaring variables: **var**, **let**, and **const**. Use a declaration keyword rather than creating an accidental global variable.

var

The lesson's key points: var can be redeclared; it is function-scoped (or global when declared outside a function); and its declarations are hoisted. A var declared inside an if block is still accessible elsewhere in the same function.

```
var a = 5;
console.log(a); // 5

var a = 8;      // redeclaration allowed
console.log(a); // 8
```

Hoisting with var

Hoisting is a simplified way to describe how JavaScript processes declarations before executing statements. With var, the declaration is available before its assignment, so the value is initially undefined.

```
console.log(a); // undefined
var a = 10;
```

A simplified mental model is: var a; console.log(a); a = 10; This is a mental model, not literal source-code movement.

let

let is block-scoped: it can be used only inside the block { } where it is declared. You cannot redeclare the same let variable in the same scope. It can be reassigned.

```
function show() {
    if (true) {
        let x = 5;
        console.log("inside if:", x); // 5
    }

    // console.log(x); // ReferenceError: x is not defined
}

show();
```

A let declaration is also hoisted in the technical sense, but it cannot be accessed before its initialization. Accessing it too early causes a ReferenceError (often described as the temporal dead zone).

```
console.log(a); // ReferenceError
let a = 10;
```

const

const is block-scoped like let, but it cannot be reassigned after initialization. A const declaration must be given a value when declared.

```
const pi = 3.14;
console.log(pi);

// pi = 4; // TypeError: reassignment is not allowed
```

For the beginner-level rule in this lesson: use let when a variable's value needs to change; use const when you do not intend to reassign it. var exists, but let/const avoid some common scope and redeclaration problems.

10. Quick comparison: var vs let vs const

Feature	var	let	const
Scope	Function/global	Block	Block
Redeclare in same scope	Yes	No	No
Reassign value	Yes	Yes	No
Access before initialization	undefined after hoisting	Error	Error
Must initialize at declaration?	No	No	Yes

11. Common confusions

Q. Does console.log show output on the webpage?
No. It shows output in Chrome DevTools Console.

Q. Does document.write show output in the Console?
No. It writes content to the webpage during parsing.

Q. Is JavaScript type-safe?
Not in the same way as a statically typed language. Variable types are dynamic; TypeScript adds static type checking.

Q. Is null the same as undefined?
No. They are different values. typeof null returns "object" because of a historical quirk.

Q. Does flex/loop/function work automatically just by defining it?
A function must be called to run. A loop runs when program flow reaches it.

Q. Why use === rather than ==?
=== checks both value and type without the implicit type conversion that == may perform.

Q. What does NaN mean?
Not a Number; it can result from an invalid numeric calculation such as "abc" - 2.

12. Practice questions

1. Which object is used to write content to the webpage?
A) console B) document C) window D) typeof

2. Where does console.log("Hi") appear?
A) webpage body B) browser alert C) DevTools Console D) page title

3. What does typeof 25 return?
A) int B) number C) integer D) numeric

4. What is the output of "5" + 2?
A) 7 B) 52 C) NaN D) Error

5. What is the output of "5" - 2?
A) 3 B) 52 C) NaN D) Error

6. Which operator checks value and type?
A) = B) == C) === D) !

7. What does the default case in switch do?
A) Always runs first B) Runs when no case matches C) Ends every loop D) Declares a variable

8. What is required to prevent an ordinary counting loop from running forever?
A) A title B) Updating the counter appropriately C) An alert D) A function return

9. What does a function help with?
A) Reusability and modularity B) Only styling C) Only HTML structure D) None

10. Which variable declaration is block-scoped and can be reassigned?
A) var B) let C) const D) none

11. Can a const variable be reassigned?
A) Always B) Never C) Only in a loop D) Only in a function

12. What happens when var a is logged before its assignment?
A) ReferenceError B) undefined C) null D) 0

13. What does NaN mean?
A) New array number B) Not a Number C) Null and Nothing D) Number as Name

14. In the nested-star example, which loop prints a single row of five stars?
A) Outer loop B) Inner loop C) switch D) function call only

15. Which statement is correct?
A) document represents the console B) console represents the browser window C) window represents the browser window D) typeof prints on the webpage

Answer key

Answer: B) document

Answer: C) DevTools Console

Answer: B) number

Answer: B) 52

Answer: A) 3

Answer: C) ===

Answer: B) Runs when no case matches

Answer: B) Updating the counter appropriately

Answer: A) Reusability and modularity

Answer: B) let

Answer: B) Never

Answer: B) undefined

Answer: B) Not a Number

Answer: B) Inner loop

Answer: C) window represents the browser window

13. Hands-on practice (try before checking notes)

1. Write an HTML page that prints “Hello World” on the page and “Hello Console” in DevTools.

2. Create a variable, print its typeof, then assign it a string and print typeof again.

3. Use the conditional operator to store the larger of two numbers in c.

4. Use an else-if ladder to map IDs 101, 102, 103 to three names; otherwise print “Not found.”

5. Print numbers 1 to 10 using a while loop, then using a for loop.

6. Use nested loops to print 4 rows of 6 stars.

7. Write a function square(n) that returns n * n, then call it and print the result.

8. Test the difference between var and let inside an if block.

9. Declare a const value and observe what happens if you try to reassign it.

10. Predict the output before running: console.log("7" + 3); console.log("7" - 3); console.log(typeof null);

14. One-page revision checklist

■ JavaScript runs in the browser through its JavaScript engine.

■ JavaScript code is written inside ....

■ document.write → webpage; console.log → DevTools Console; window.alert → popup.

■ Use typeof to check a value's type.

■ Arithmetic: + - * / % **; relational: < > <= >=; equality: == != ===; logical: || && !.

■ Conditional operator: condition ? value1 : value2.

■ if-else handles decisions; else-if ladder checks alternatives; switch compares cases.

■ while checks a condition before each repetition; for groups initialization, condition, update.

■ Nested loops can repeat a row/column pattern.

■ Functions group statements; they support reusability and modularity.

■ var: function-scoped, redeclarable, hoisted; let: block-scoped, no same-scope redeclaration; const: block-scoped, no reassignment.

■ NaN means Not a Number; typeof null returns object as a historical quirk.